

Identification *in silico* d'une séquence codante

(Durée : 30 minutes, sur 5 points – avec un barème sur... plus de points!)

Lors de l'annotation d'un génome, on analyse le brin codant (non transcrit) de l'ADN, orienté de 5' vers 3'. L'identification d'une Séquence Codante (CDS) repose sur la détection d'une Phase Ouverte de Lecture (ORF).

Celle-ci débute par un codon d'initiation ATG et s'achève au premier codon de terminaison rencontré (pour simplifier ici, on se limitera au codon stop TAA) situé dans le même cadre de lecture (c'est-à-dire que le nombre de nucléotides les séparant doit être un multiple de 3).

On travaillera sur la séquence suivante :

```
brin_codant = "GGCATGGTCAGTCGCTACTAATGC"
               0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
```

Question 1 : Repérage d'un codon

Écrivez une fonction `cherche_codon(adn, codon)` qui parcourt la chaîne de caractères `adn` et renvoie l'indice de la première occurrence du codon cherché. Si le codon n'est pas présent, la fonction devra renvoyer `False`.

NB : cette question sur 2 points est identique (à l'habillage près) à un exercice vu en TP.

```
def cherche_codon(adn, codon):
    for i in range(len(adn) - len(codon) + 1):
        if adn[i:i + len(codon)] == codon:
            return i
    return False
```

⚠ range ne doit pas causer des problèmes

⚠ il faut bien couper adn!

Question 2 : Application

En utilisant la fonction définie à la question précédente, écrivez les deux commandes Python permettant de trouver respectivement l'indice du codon start (ATG) et l'indice du codon stop (TAA) dans la séquence `brin_codant`. Précisez également les valeurs numériques qui seront renvoyées par l'exécution de ces deux commandes.

```
print(cherche_codon(brin_codant, "ATG"))
>>> 3

print(cherche_codon(brin_codant, "TAA"))
>>> 18
```

Question 3 : Extraction de la CDS

En utilisant les fonctions précédentes et le "slicing" de chaînes, écrivez une fonction `extraction_cds(adn)` qui renvoie la séquence d'ADN de l'ORF complète, incluant le codon start et le codon stop.

```
def extraction_cds(adm):
    i_start = cherche_codon(adm, "ATG")
    i_stop = cherche_codon(adm, "TAA")

    if i_start is False or i_stop is False or i_stop < i_start:
        return False

    return adm[i_start:i_stop+3]
```

⚠ si i_start = 0, alors i_start == False donne True

⚠ i_stop est le début du codon !

Question 4 : Application de l'extraction

Écrivez la commande Python permettant d'extraire la CDS de la séquence `brin_codant` en utilisant la fonction définie à la question 3. Précisez la chaîne de caractères exacte qui sera renvoyée par cette commande.

```
extraction_cds(brin_codant)

>>> "ATGGTCAGTCGCTACTAA"
```

Question 5 : Mutation (Question difficile, en bonus !)

On remarque qu'une mutation classique de cet ADN prend le codon `CGC` et le transforme en `CAC`. Faites la fonction `mutation(cds, codon, changement)` qui cherche le premier codon correspondant et le remplace par son changement (une seule fois). **ATTENTION** : Il faut respecter le cadre de lecture (la règle des 3). Le remplacement ne doit avoir lieu que si le codon à muter est en phase (son indice dans la séquence doit être un multiple de 3).

```
def mutation(cds, codon, changement):
    for i in range(0, len(cds) - len(codon) + 1, 3):
        if cds[i:i+len(codon)] == codon:
            return cds[:i] + changement + cds[i+len(codon):]

    return cds
```

⚠ range peut faire des sauts de 3

⚠ "a" + "b"
>>> "ab"